Unit 3 · Making decisions in Python

CONCEPT

Unit 2 calculates values. Unit 3 uses those values to select different actions. Learn how a computer decides which statements to execute.

EXAMPLE / TAKEAWAY

Learning outcomes: evaluate comparisons; trace Boolean conditions; construct if, elif and else; combine conditions; test boundary cases.

A program can choose

CONCEPT

A sequence always follows one path. A selection structure follows different paths depending on whether a condition is True or False.

EXAMPLE / TAKEAWAY

Example: if the allowance balance is negative, display a warning; otherwise display an appropriate alternative.

Decision-making starts with a question

CONCEPT

Write a question that can be answered True or False. Use a comparison as the condition and connect each answer to an action.

EXAMPLE / TAKEAWAY

Balance < 0?

True → Over budget

False → Check the remaining cases.

Boolean values: True and False

CONCEPT

The bool type represents logical truth values. Comparison expressions usually produce Boolean results. Python spells these values True and False.

EXAMPLE / PYTHON

```
days = 5
print(days > 0)    # True
print(type(days > 0))  # bool
```

Comparison: == and !=

CONCEPT

Use == to ask whether two values are equal; use != to ask whether they differ. A single = assigns a value instead of comparing.

EXAMPLE / PYTHON

```
print(5 == 5)  # True
print(5 != 3)  # True
print(5 == 3)  # False
```

Comparison: > and <

CONCEPT

Use > for greater than and < for less than. These comparisons are useful for checking a positive balance or detecting a negative value.

EXAMPLE / PYTHON

```
balance = -2000
print(balance < 0)  # True
print(balance > 0)  # False
```

Comparison: >= and <=

CONCEPT

Use >= and <= to include equality in a threshold. A boundary such as exactly 7 days should be tested deliberately.

EXAMPLE / PYTHON

```
days = 7
print(days <= 7)  # True
print(days < 7)   # False
```

Assignment and comparison are different

CONCEPT

A single equals sign stores a value. A double equals sign checks whether two values are equal. Confusing them can produce invalid code.

EXAMPLE / PYTHON

```
balance = 0   # assignment
print(balance == 0)  # True

# if balance = 0:  <- invalid
```

The if statement

CONCEPT

An if statement checks its condition. When it is True, Python executes the indented block. When it is False, Python skips that block.

EXAMPLE / PYTHON

```
temperature = 34
if temperature > 30:
    print("Hot day")
```

Why the colon and indentation matter

CONCEPT

The colon introduces the block. Indentation defines which statements belong to it. Four spaces per level is the usual Python convention.

EXAMPLE / PYTHON

```
balance = -2000
if balance < 0:
    print("Warning")
    print("Review your spending")
```

Statements outside the block

CONCEPT

Statements aligned with if are not conditional. They run after the if block whether or not the condition was true.

EXAMPLE / PYTHON

```
balance = 1500
if balance < 0:
    print("Over budget")
print("Check complete")  # Always
```

Two alternative paths: if and else

CONCEPT

Use else when one action is required if the condition is True and another is required if it is False. Only one block executes.

EXAMPLE / PYTHON

```
amount = 4000
if amount >= 5000:
    print("Example discount")
else:
    print("No discount")
```

Trace a two-way decision

CONCEPT

Evaluate the condition before predicting the output. Then identify the single branch that runs. Avoid assuming both if and else execute.

EXAMPLE / TAKEAWAY

amount = 6000: amount >= 5000 → True

Output: Example discount

amount = 4000: condition → False

Output: No discount

Three-way decisions: elif

CONCEPT

Use elif for an additional condition. Python checks from top to bottom and executes the first matching branch. The final else covers remaining cases.

EXAMPLE / PYTHON

```
if balance < 0:
    print("Over budget")
elif balance == 0:
    print("Exactly on budget")
else:
    print("Within budget")
```

Negative, zero and positive balances

CONCEPT

An allowance balance can fall into one of three non-overlapping groups. A three-way decision reports a meaningful status for each group.

EXAMPLE / TAKEAWAY

-2000 → Over budget

0 → Exactly on budget

7500 → Within budget

Why condition order matters

CONCEPT

When conditions overlap, an earlier broad condition may hide a later more specific condition. Place more specific thresholds first.

EXAMPLE / PYTHON

```
score = 90
if score >= 80:
    print("High achievement")
elif score >= 50:
    print("Basic target achieved")
```

The and operator

CONCEPT

A and B is True only when both A and B are True. It is useful for checking whether a value is inside an allowed interval.

EXAMPLE / PYTHON

```
days = 5
if days >= 0 and days <= 7:
    print("Accepted")
```

Truth table for and

CONCEPT

A and B is false when either component is false. Practice predicting this result before running the expressions in Python.

EXAMPLE / PYTHON

```
True  and True  → True
True  and False → False
False and True  → False
False and False → False
```

The or operator

CONCEPT

A or B is True when at least one condition is True. This can detect several different invalid-input situations with one message.

EXAMPLE / PYTHON

```
food = -100
transport = 2000
if food < 0 or transport < 0:
    print("Invalid cost")
```

Truth table for or

CONCEPT

A or B is false only when both A and B are false. This is different from and, which needs both conditions to be true.

EXAMPLE / PYTHON

```
True  or True  → True
True  or False → True
False or True  → True
False or False → False
```

The not operator

CONCEPT

not reverses a Boolean value. Use clear variable names such as is_open and has_data to communicate what the Boolean means.

EXAMPLE / PYTHON

```
is_open = False
if not is_open:
    print("Closed")

print(not True)  # False
```

Chained comparisons

CONCEPT

Python can express numerical range checks directly. The chained form below checks both boundaries and is equivalent to using and.

EXAMPLE / PYTHON

```
days = 7
print(0 <= days <= 7)  # True
print(0 <= 8 <= 7)     # False
```

Nested decisions

CONCEPT

An if can appear inside another if. The inner choice happens only when execution reaches that outer branch; each nesting level is indented.

EXAMPLE / PYTHON

```
if days > 0:
    if balance >= 0:
        print("Non-negative balance")
    else:
        print("Over budget")
```

First plan with a decision table

CONCEPT

A decision table lists the condition and the required action. It helps avoid missing a case or assigning contradictory messages.

EXAMPLE / TAKEAWAY

Balance < 0 → Over budget

Balance == 0 → Exactly on budget

Otherwise → Within budget

Translate the table into pseudocode

CONCEPT

Pseudocode expresses the logic in clear steps before worrying about exact Python syntax. Compare the planned branches to the actual if / elif / else.

EXAMPLE / TAKEAWAY

IF balance is negative: warn

ELSE IF balance is zero: exact

ELSE: positive remaining allowance

The budget problem: inputs

CONCEPT

Extend the Unit 2 programme with a choice of three budget messages. Ask for daily food, daily transport, number of campus days and allowance.

EXAMPLE / PYTHON

```
food = float(input("Food/day: "))
transport = float(input("Transport/day:
"))
days = int(input("Campus days: "))
allowance = float(input("Allowance: "))
```

The budget calculation

CONCEPT

For valid inputs, use the same arithmetic as Unit 2. The difference is what happens after the balance has been calculated.

EXAMPLE / PYTHON

```
weekly_cost = (food + transport) * days
balance = allowance - weekly_cost
print(f"Cost: TZS {weekly_cost:.2f}")
print(f"Balance: TZS {balance:.2f}")
```

Reject values outside the valid range

CONCEPT

Conditions can check range rules before ordinary calculation. In this example, costs and allowance cannot be negative and campus days must lie from 0 to 7.

EXAMPLE / PYTHON

```
invalid = (food < 0 or transport < 0
           or allowance < 0
           or not (0 <= days <= 7))
if invalid:
    print("Invalid values")
```

Choose the message

CONCEPT

After passing validation, use a three-way decision to describe the result. Explain why exactly zero needs its own branch.

EXAMPLE / PYTHON

```
if balance < 0:
    print("Over budget")
elif balance == 0:
    print("Exactly on budget")
else:
    print("Within budget")
```

Trace example: within budget

CONCEPT

Food 4000 and transport 2500 over five days cost 32500. From an allowance of 40000, the balance is 7500, which is positive.

EXAMPLE / TAKEAWAY

(4000 + 2500) × 5 = 32500

40000 − 32500 = 7500

Status → Within budget

Trace example: over budget

CONCEPT

Food 3500 and transport 2000 over four days cost 22000. From an allowance of 20000, the balance is -2000.

EXAMPLE / TAKEAWAY

(3500 + 2000) × 4 = 22000

20000 − 22000 = −2000

Status → Over budget

Trace example: exactly on budget

CONCEPT

Food 2500 and transport 1500 over five days cost 20000. If the allowance is also 20000, the balance is exactly zero.

EXAMPLE / TAKEAWAY

(2500 + 1500) × 5 = 20000

20000 − 20000 = 0

Status → Exactly on budget

Boundary tests are not optional

CONCEPT

A typical test alone cannot prove all branches work. Test zero, negative and positive results as well as invalid values and edge cases.

EXAMPLE / TAKEAWAY

Days = 0 → valid, zero cost

Days = 7 → valid upper limit

Days = 8 → invalid input

Balance = 0 → exact-budget branch

Find and fix common mistakes

CONCEPT

Look for a missing colon, inconsistent indentation, the wrong comparison operator, a misplaced branch or a text value compared with a number.

EXAMPLE / TAKEAWAY

Incorrect: if balance = 0:

Correct: if balance == 0:

Incorrect boundary: days < 7

Correct range: 0 <= days <= 7

Independent conditions versus exclusive alternatives

CONCEPT & REASONING

Separate if statements are independent: each condition is evaluated when execution reaches it. An if/elif/else chain is exclusive: only the first true branch runs. The design depends on whether the requirements permit several outcomes or exactly one.

EXAMPLE & RESULT

```
score = 85
if score >= 50:
    print("Pass")
if score >= 80:
    print("Distinction")
```

Both messages appear. In an if/elif chain, testing score >= 80 first would produce only Distinction.

A decision flowchart

CONCEPT & REASONING

A diamond represents a question with true and false paths. A rectangle represents an action. A decision chart can show which paths meet again before execution continues. Indentation in Python represents the statements belonging to each path.

EXAMPLE & RESULT

```
Start → score >= 80?
True → Distinction → End
False → score >= 50?
  True → Pass → End
  False → Fail → End
```

A score of 79 follows the second question; a score of 80 follows the first true path.

Rules, ranges and order

CONCEPT & REASONING

An exclusive classification uses disjoint intended ranges. Overlapping tests are resolved by branch order, so the highest threshold is checked first when the condition is >=. A valid score lies from 0 to 100 inclusive; an invalid score is not a failing grade.

EXAMPLE & RESULT

```
score = 101
if not 0 <= score <= 100:
    print("Invalid")
elif score >= 80:
    print("Distinction")
elif score >= 50:
    print("Pass")
else:
    print("Fail")
```

101 is Invalid. 49 is Fail, 50 is Pass, 79 is Pass, 80 is Distinction.

Short-circuit evaluation

CONCEPT & REASONING

Python evaluates and from left to right and stops when the left operand is false; or stops when the left operand is true. This can protect an operation that is meaningful only when an earlier condition holds.

EXAMPLE & RESULT

```
total = 0
count = 0
if count > 0 and total / count > 10:
    print("Above target")
else:
    print("Not above target")
```

Not above target. Division is not evaluated when count > 0 is false.

Rule-based classification in agriculture

CONCEPT & REASONING

A rule-based system classifies values according to explicitly defined conditions. The example uses fictional percentage thresholds to illustrate a programming decision, not a plant-disease diagnosis. A machine-learning model instead learns patterns from data; its predictions still need validation.

EXAMPLE & RESULT

```
affected = 35
if not 0 <= affected <= 100:
    print("Invalid")
elif affected >= 50:
    print("High")
elif affected >= 20:
    print("Moderate")
else:
    print("Low")
```

35 is Moderate. Tests at 19, 20, 49, 50 and invalid values reveal whether the rules match their specification.

Practice lab and review

CONCEPT

Predict outputs for ordinary and boundary cases. Follow the coloured branch animation and check explanations. Then reproduce the logic in a Python editor.

EXAMPLE / TAKEAWAY

Browser illustrations demonstrate branching; they do not execute Python. Use your own Python environment to test real code.

Independent Activity U3

CONCEPT

Create a weekly budget adviser based on Unit 2. Add input-range checking and three outcomes. Submit the runnable Python file and short notes with five test runs.

EXAMPLE / TAKEAWAY

Evidence: algorithm + .py file + expected/actual test table + brief explanation and assistance declaration.

Before Unit 4: repeating actions

CONCEPT

Unit 3 uses comparisons and conditions to choose an action. Unit 4 will repeat actions with for and while loops. Complete the Unit 3 activity before moving on.

EXAMPLE / TAKEAWAY

You should now explain True / False, if / elif / else, and / or / not, indentation, branch order and boundary testing.